

12 – List Comprehension, Functions as Objects, Testing, Debugging

1. List Comprehensions (Comprensiones de listas)

¿Qué es?

Una forma concisa de construir listas aplicando una función a cada elemento de una secuencia, opcionalmente filtrando con una condición.

Forma general:

```
1 [expresión for elemento in iterable if condición]
```

Ejemplo tradicional vs comprensión:

```
1 # Tradicional
2 def f(L):
3     Lnew = []
4     for e in L:
5         if e % 2 == 0:
6             Lnew.append(e ** 2)
7     return Lnew
8
9 # Con comprensión
10 Lnew = [e ** 2 for e in L if e % 2 == 0]
```

Ejercicio:

```
1 [len(x) for x in ['xy', 'abcd', 7, '4.0'] if type(x) == str]
2 # Resultado: [2, 4, 3]
```

2. Funciones con parámetros por defecto



Ejemplo: raíz cuadrada por bisección

```
1 def bisection_root(x, epsilon=0.01):
2     low = 0
3     high = x
4     guess = (high + low)/2.0
5     while abs(guess**2 - x) >= epsilon:
6         if guess**2 < x:
7             low = guess
8         else:
9             high = guess
10        guess = (high + low)/2.0
11    return guess
```



Reglas para parámetros por defecto:

- Los parámetros con valores por defecto deben ir **al final**.
- Ejemplos válidos:

```
1 bisection_root(123)
2 bisection_root(x = 123, epsilon = 0.001)
```

- Ejemplos inválidos:

```
1 bisection_root(epsilon = 0.001, 123) # ✗
```



3. Funciones como objetos (y funciones que devuelven funciones)



¿Qué significa?

- Las funciones en Python son objetos de primera clase: pueden asignarse a variables, pasarse como argumentos, o devolverse desde otras funciones.



Ejemplo:

```
1 def make_prod(a):
2     def g(b):
3         return a * b
```

```
4     return g
5
6     doubler = make_prod(2)
7     print(doubler(3)) # 6
```

Explicación de ámbito (scope):

- `g` está definida dentro de `make_prod`, y captura la variable `a`.
- Cada vez que llamás a `make_prod`, se crea una nueva versión de `g` con su propio valor de `a`.

4. Testing y debugging (pruebas y depuración)

Odiarnos sacar este tema, pero el *Dr. Pangloss* estaba equivocado. No vivimos en «el mejor de los mundos posibles». Hay lugares en los que llueve demasiado poco y otros en los que llueve demasiado. Algunos lugares son demasiado fríos, otros demasiado calurosos, y otros demasiado calurosos en verano y demasiado fríos en invierno. A veces la bolsa baja mucho. A veces los tramposos ganan. Y, por si fuera poco, *nuestros programas no siempre funcionan correctamente la primera vez que los ejecutamos*.

Testear es el *proceso de ejecutar un programa para intentar determinar si funciona como está previsto*. *Depurar* es el *proceso de intentar arreglar un programa que ya se sabe que no funciona como se pretendía*.

Las pruebas y la depuración no son procesos en los que se deba empezar a pensar después de construir un programa. Los buenos programadores diseñan sus programas de forma que sea más fácil probarlos y depurarlos. La clave para hacer esto es dividir el programa en componentes separados que puedan ser implementados, probados y depurados independientemente de otros componentes.

El *primer paso para que un programa funcione es conseguir que el sistema del lenguaje acepte ejecutarlo*, es decir, eliminar los errores de sintaxis y los errores semánticos estáticos que pueden detectarse sin ejecutar el programa.

Testing

El objetivo de las pruebas es demostrar que existen fallos, no que un programa está libre de ellos. Citando a *Edsger Dijkstra*, «Las pruebas de programas pueden usarse para mostrar la presencia de fallos, ¡pero nunca para mostrar su ausencia!». O, como supuestamente dijo *Albert Einstein*: «Ninguna cantidad de experimentos puede demostrar que tengo razón; un solo experimento puede demostrar que estoy equivocado».

¿Por qué? Incluso el programa más sencillo tiene miles de millones de entradas posibles. Consideremos, por ejemplo, un programa que pretende cumplir la especificación

```
1 def is_smaller(x, y):
2     """Assumes x and y are ints
3     Returns True if x is less than y and False otherwise."""
```

Ejecutarlo en todos los pares de enteros sería, como mínimo, tedioso. Lo mejor que podemos hacer es ejecutarlo en pares de enteros que tengan una probabilidad razonable de producir una respuesta incorrecta si hay un error en el programa.

La clave de las pruebas es encontrar una colección de entradas, denominada «*testsuite*», que tenga una alta probabilidad de revelar errores y, al mismo tiempo, no tarde demasiado en ejecutarse. La clave es dividir el espacio de todas las entradas posibles en subconjuntos que proporcionen información equivalente sobre la corrección del programa y, a continuación, construir un conjunto de pruebas que contenga al menos una entrada de cada partición. (Normalmente, no es posible construir un conjunto de pruebas de este tipo. Piense en esto como un ideal inalcanzable).

Una partición de un conjunto divide ese conjunto en una colección de subconjuntos tales que cada elemento del conjunto original pertenece exactamente a uno de los subconjuntos. Consideremos, por ejemplo, `is_smaller(x, y)`. El conjunto de entradas posibles son todas las combinaciones pareadas de números enteros. Una forma de dividir este conjunto es en estos nueve subconjuntos:

```
1 x positive, y positive, x < y
2 x positive, y positive, y < x
3 x negative, y negative, x < y
4 x negative, y negative, y < x
5 x negative, y positive
6 x positive, y negative
7 x = 0, y = 0
8 x = 0, y ≠ 0
9 x ≠ 0, y = 0
```

Si probáramos la implementación en al menos un valor de cada uno de estos subconjuntos, tendríamos una buena oportunidad (pero no garantía) de exponer un error si es que existe.

Para la mayoría de los programas, encontrar una buena partición de las entradas es mucho más fácil de decir que de hacer. Por lo general, se recurre a heurísticas basadas en la exploración de distintos caminos a través de alguna combinación del código y las especificaciones. La heurística basada en la *exploración de caminos a través del código* pertenece a una clase denominada *prueba de caja de cristal* (o caja blanca). La heurística basada en *explorar caminos a través de la especificación* pertenece a la clase denominada *pruebas de caja negra*.

Black-Box Testing

En principio, *las pruebas de caja negra se construyen sin mirar el código que se va a probar*. Las pruebas de caja negra permiten que los probadores y los ejecutores provengan de poblaciones separadas. Cuando los que impartimos cursos de programación generamos casos de prueba para los conjuntos de problemas que asignamos a los alumnos, estamos desarrollando conjuntos de pruebas de caja negra. Los desarrolladores de software comercial suelen tener grupos de control de calidad que son en gran medida independientes de los grupos de desarrollo. Ellos también desarrollan conjuntos de pruebas de caja negra.

Esta independencia reduce la probabilidad de generar conjuntos de pruebas que presenten errores correlacionados con errores en el código. Supongamos, por ejemplo, que el autor de un programa asume de forma implícita, pero no válida, que nunca se llamará a una función con un número negativo. Si la misma persona construyera el conjunto de pruebas para el programa, probablemente repetiría el error y no probaría la función con un argumento negativo.

Otra característica positiva de las pruebas de caja negra es que *son robustas con respecto a los cambios de implementación*. Como los datos de las pruebas se generan sin conocer la implementación, no es necesario modificar las pruebas cuando se cambia la implementación.

Como hemos dicho antes, una buena forma de generar datos de prueba de caja negra es explorar caminos a través de una especificación. Consideremos, la especificación

```
1 def sqrt(x, epsilon):
2     """Assumes x, epsilon floats
```

```

3         x >= 0
4         epsilon > 0
5     Returns result such that
6         x - epsilon <= result * result <= x + epsilon"""

```

Parece que sólo hay dos caminos distintos a través de esta especificación: uno correspondiente a $x = 0$ y otro correspondiente a $x > 0$. Sin embargo, el sentido común nos dice que aunque es necesario probar estos dos casos, difícilmente es suficiente.

También *deben comprobarse las condiciones límite*. Examinar un argumento de tipo lista a menudo significa examinar la lista vacía, una lista con exactamente un elemento, una lista con elementos inmutables, una lista con elementos mutables y una lista que contiene listas. Cuando se trata de números, normalmente significa mirar valores muy pequeños y muy grandes, así como valores «típicos». Para `sqrt`, por ejemplo, podría tener sentido probar valores de `x` y `epsilon` similares a los de la siguiente tabla.

Prueba de las condiciones límite

x	epsilon
0.0	0.0001
25.0	0.0001
0.5	0.0001
2.0	0.0001
2.0	1.0/2.0 ** 64.0
1.0/2.0 ** 64	1.0/2.0 ** 64.0
2.0 ** 64.0	1.0/2.0 ** 64.0
1.0/2.0 ** 64.0	2.0 ** 64.0
2.0 ** 64.0	2.0 ** 64.0

Las cuatro primeras filas pretenden representar casos típicos. Observe que los valores de `x` incluyen un cuadrado perfecto, un número menor que uno y un número con raíz cuadrada irracional. Si alguna de estas pruebas falla, hay un error en el programa que necesita ser corregido.

Las filas restantes prueban valores extremadamente grandes y pequeños de `x` y `epsilon`. Si alguna de estas pruebas falla, es que hay que arreglar algo. Quizá haya un error en el código que haya que corregir, o quizá haya que cambiar la especificación para que sea más fácil de cumplir. Por ejemplo, puede que no sea razonable esperar encontrar una

aproximación de una raíz cuadrada cuando `epsilon` es ridículamente pequeño.

Otra condición de contorno importante es el `aliasing`. Consideremos el código

```
1 def copy(L1, L2):
2     """Assumes L1, L2 are lists Mutates L2 to be a copy of L1"""
3     while len(L2) > 0: #remove all elements from L2
4         L2.pop() #remove last element of L2
5
6     for e in L1: #append L1's elements to initially empty L2
7         L2.append(e)
```

Funcionará la mayoría de las veces, pero no cuando L1 y L2 hagan referencia a la misma lista. Cualquier conjunto de pruebas que no incluyera una llamada de la forma `copy(L, L)`, no revelaría el fallo.

Glass-Box Testing

Las pruebas de caja negra nunca deben omitirse, pero rara vez son suficientes. Sin examinar la estructura interna del código, es imposible saber qué casos de prueba pueden aportar información nueva. Consideremos un ejemplo trivial:

```
1 def is_prime(x):
2     """Assumes x is a nonnegative int
3     Returns True if x is prime; False otherwise"""
4     if x <= 2:
5         return False
6     for i in range(2, x):
7         if x % i == 0:
8             return False
9     return True
```

Si observamos el código, podemos ver que, debido a la prueba si `x <= 2`, los valores `0`, `1` y `2` se tratan como casos especiales y, por lo tanto, deben probarse. Sin mirar el código, uno podría no probar `is_prime(2)`, y por lo tanto no descubriría que la llamada a la función `is_prime(2)` devuelve `False`, indicando erróneamente que `2` no es un primo.

Las suites de pruebas de caja de cristal suelen ser mucho más fáciles de construir que las de caja negra. Las especificaciones suelen estar incompletas y a menudo bastante descuidadas, por lo que resulta difícil estimar hasta qué punto un conjunto de pruebas de caja negra explora el espacio de entradas interesantes. En cambio, la noción de

camino a través del código está bien definida, y es relativamente fácil evaluar hasta qué punto se explora el espacio. De hecho, existen herramientas comerciales que permiten medir objetivamente la exhaustividad de las pruebas de caja de cristal.

Un conjunto de pruebas **glass-box** es **path-complete** (camino completo) si ejerce todos los posibles caminos a través del programa. Esto suele ser imposible de conseguir, porque depende del número de veces que se ejecute cada bucle y de la profundidad de cada recursión. Por ejemplo, una implementación recursiva de factorial sigue un camino diferente para cada entrada posible (porque el número de niveles de recursión será diferente).

Además, ni siquiera un conjunto de pruebas completo garantiza que se detecten todos los errores.

```
1 def abs(x):
2     """Assumes x is an int
3     Returns x if x >= 0 and -x otherwise"""
4     if x < -1:
5         return -x
6     else:
7         return x
```

La especificación sugiere que hay dos casos posibles: `x` es negativo o no lo es. Esto sugiere que el conjunto de entradas `{2, -2}` es suficiente para explorar todos los caminos de la especificación. Este conjunto de pruebas tiene la agradable propiedad adicional de forzar al programa a recorrer todos sus caminos, por lo que también parece un conjunto de pruebas **glass-box** completo. El único problema es que este conjunto de pruebas no expondrá el hecho de que `abs(-1)` devolverá `-1`.

A pesar de las limitaciones de las pruebas **glass-box**, suele merecer la pena seguir algunas reglas generales:

- Ejecute ambas ramas de todas las sentencias `if`.
- Asegúrese de que se ejecuta cada cláusula `except` (véase el capítulo 9).
- Para cada bucle `for`, disponga de casos de prueba en los que
 - No se entre en el bucle (por ejemplo, si el bucle está iterando sobre los elementos de una lista, asegúrese de que se prueba sobre la lista vacía).
 - El cuerpo del bucle se ejecuta exactamente una vez.
 - El cuerpo del bucle se ejecuta más de una vez.
- Para cada bucle `while`

- Observe los mismos tipos de casos que cuando se trata de bucles `for`.
- Incluya casos de prueba correspondientes a todas las formas posibles de salir del bucle. Por ejemplo, para un bucle que comienza con `while len(L) > 0 and not L[i] == e` encuentre casos en los que el bucle sale porque `len(L)` es mayor que cero y casos en los que sale porque `L[i] == e`.
- Para funciones recursivas, incluya casos de prueba que hagan que la función retorne sin llamadas recursivas, exactamente con una llamada recursiva y con más de una llamada recursiva.

Conducting Tests

A menudo se piensa que las pruebas se realizan en dos fases. **Siempre hay que empezar por las pruebas unitarias**. Durante esta fase, los testers construyen y ejecutan pruebas diseñadas para determinar si las unidades individuales de código (por ejemplo, funciones) funcionan correctamente. A continuación se realizan las **pruebas de integración**, destinadas a comprobar si grupos de unidades funcionan correctamente cuando se combinan. Por último, las **pruebas funcionales** se utilizan para comprobar si el programa en su conjunto se comporta según lo previsto. En la práctica, los responsables de las pruebas recorren estas fases cíclicamente, ya que los fallos durante las pruebas de integración o funcionales obligan a introducir cambios en las unidades individuales.

Las **pruebas funcionales** son casi siempre la fase más difícil. El comportamiento previsto de todo un programa es considerablemente más difícil de caracterizar que el comportamiento previsto de cada una de sus partes. Por ejemplo, caracterizar el comportamiento previsto de un procesador de textos es considerablemente más difícil que caracterizar el comportamiento del subsistema que cuenta el número de caracteres de un documento. Los problemas de escala también pueden dificultar las pruebas funcionales. *No es raro que las pruebas funcionales tarden horas o incluso días en ejecutarse.*

Muchas organizaciones de desarrollo de software industrial tienen un **grupo de aseguramiento de la calidad del software** (SQA) que es independiente del grupo encargado de implementar el software. La misión de un grupo de SQA es garantizar que, antes de que el software salga al mercado, sea adecuado para el fin previsto. En algunas organizaciones, el grupo de desarrollo se encarga de las pruebas

unitarias y *el grupo de garantía de calidad, de las pruebas de integración y funcionales.*

En la industria, el proceso de pruebas suele estar muy automatizado. Los evaluadores no se sientan ante terminales para teclear entradas y comprobar salidas. En su lugar, utilizan controladores de pruebas.

- Configurar el entorno necesario para invocar el programa (o las unidades) a probar.
- Invocar el programa (o las unidades) a probar con una secuencia de entradas predefinida o generada automáticamente.
- Guardar los resultados de estas invocaciones.
- Comprobar la aceptabilidad de los resultados de las pruebas.
- Preparar un informe apropiado.

¿Qué es un *stub*?

Un *stub* es una función o módulo *falso* que simula el comportamiento de una parte del código que todavía no existe, es muy costosa de ejecutar o es innecesaria para el test actual.

Existen terminos parecidos:

- *Stub*: reemplaza una función y devuelve algo fijo.
- *Mock*: además de simular, *verifica* si fue llamada, cuántas veces, con qué argumentos, etc.
- *Fake*: implementación completa pero simplificada (como una base de datos en memoria).
- *Dummy*: un valor que se pasa pero que no se usa.

Durante las *pruebas unitarias*, a menudo necesitamos construir *stubs* además de *drivers*. Los controladores simulan partes del programa que utilizan la unidad que se está probando, mientras que los *stubs* simulan partes del programa utilizadas por la unidad que se está probando. Los *stubs* son útiles porque permiten probar unidades que dependen de software o, a veces, incluso de hardware que aún no existe. Esto permite a los equipos de programadores desarrollar y probar simultáneamente varias partes de un sistema. Idealmente, un *stub* debería

- Comprobar la razonabilidad del entorno y los argumentos suministrados por el llamante (llamar a una función con argumentos inapropiados es un error común).

- Modificar los argumentos y las variables globales de forma coherente con la especificación.
- Devolver valores coherentes con la especificación.

Construir stubs adecuados suele ser todo un reto. Si la unidad a la que sustituye el stub debe realizar alguna tarea compleja, crear un stub que realice acciones coherentes con la especificación puede equivaler a escribir el programa que el stub pretende sustituir. Una forma de superar este problema es limitar el conjunto de argumentos aceptados por el stub, y crear una tabla que contenga los valores a devolver para cada combinación de argumentos a utilizar en el conjunto de pruebas.

Uno de los atractivos de automatizar el proceso de pruebas es que facilita las pruebas de regresión. Cuando los programadores intentan depurar un programa, es demasiado habitual instalar un «arreglo» que rompe algo, o quizá muchas cosas, que antes funcionaban. Cada vez que se realiza un cambio, por pequeño que sea, hay que comprobar que el programa sigue superando todas las pruebas que solía superar.

Debugging

Existe una encantadora leyenda urbana sobre cómo el proceso de corregir fallos en el software llegó a conocerse como depuración. La foto es de una página, del 9 de septiembre de 1947, de un libro de laboratorio del grupo que trabajaba en la **Mark II Aiken Relay Calculator** de la Universidad de Harvard. Obsérvese la polilla pegada a la página y la frase «*Primer caso real de bug encontrado*» debajo de ella.

0800 Unitan started
 1000 " stopped - antam ✓
 1300 (033) MP - MC 1.582142000
 (033) PRO 2 2.130476415
 coned 2.130676415
 Relays 6-2 in 033 failed special speed test
 in relay " 11.00 test.
 Relays changed
 1100 Started Cosine Tape (Sine check)
 1525 Started Multy Adder Test.
 1545  Relay #70 Panel F
 (moth) in relay.
 First actual case of bug being found.
 1630 antam started.
 1700 closed down.

1.2700 9.037 847 025
 9.037 846 995 coned
 4.615925059(-2)
 Relay 3370

Algunos han afirmado que el descubrimiento de esa desafortunada polilla atrapada en el **Mark II** dio lugar al uso de la frase «**Debugging**». Sin embargo, la expresión «**primer caso real de descubrimiento de un bug**» sugiere que ya era común una interpretación menos literal de la frase. **Grace Murray Hopper**, una de las responsables del proyecto **Mark II**, dejó claro que el término «**bug**» ya se utilizaba ampliamente para describir los problemas de los sistemas electrónicos durante la Segunda Guerra Mundial.

Y mucho antes de eso, el **New Catechism of Electricity** de **Hawkins**, un manual de electricidad de 1896, incluía la entrada: «**El término "bug" se utiliza de forma limitada para designar cualquier fallo o problema en las conexiones o el funcionamiento de los aparatos eléctricos**». En inglés, la palabra «**bugbear**» significa «**cualquier cosa que causa miedo o ansiedad aparentemente innecesarios o excesivos**». Parece que **Shakespeare** lo acertó a «**bug**» cuando hizo que **Hamlet** se quejara de «**bugs and goblins in my life**».

El uso de la palabra «**bug**» a veces lleva a la gente a ignorar el hecho fundamental de que si has escrito un programa y tiene un «**bug**», lo has estropeado. Los errores no se cuelan de improviso en programas perfectos. **Si tu programa tiene un error, es porque tú lo pusiste ahí**. Los errores no se reproducen en los programas. Si su programa tiene varios fallos, es porque usted ha cometido varios errores.

Los errores de ejecución pueden clasificarse en dos categorías:

- **Evidente → encubierto**: Un **bug evidente** tiene una manifestación obvia; por ejemplo, el programa se bloquea o tarda mucho más (quizás para siempre) en ejecutarse de lo que debería. Un **bug encubierto** no tiene una manifestación obvia. El programa puede ejecutarse hasta el final sin problemas... salvo que entrega una respuesta incorrecta. Muchos bugs están en algún punto entre estos dos extremos, y el que un bug sea evidente o no puede depender de cuán cuidadosamente examines el comportamiento del programa.
- **Persistente → intermitente**: Un **bug persistente** ocurre *cada vez* que se ejecuta el programa con los mismos datos de entrada. Un **bug intermitente** ocurre solo *a veces*, incluso cuando el programa se ejecuta con las mismas entradas y, aparentemente, bajo las mismas condiciones.

El mejor tipo de fallos son los **manifiestos y persistentes**. Los desarrolladores no pueden hacerse ilusiones sobre la conveniencia de desplegar el programa. Y si alguien es tan tonto como para intentar usarlo, descubrirá rápidamente su insensatez. Tal vez el programa haga algo horrible antes de colapsar, por ejemplo, borrar archivos, pero al menos el usuario tendrá motivos para preocuparse (si no entrar en pánico). Los buenos programadores intentan escribir sus programas de forma que los errores de programación provoquen fallos que sean manifiestos y persistentes. Esto suele denominarse **programación defensiva**.

El siguiente paso en el pozo de lo indeseable son los errores evidentes pero intermitentes. Un sistema de control del tráfico aéreo que calcula la ubicación correcta de los aviones casi todo el tiempo sería mucho más peligroso que uno que comete errores evidentes todo el tiempo. Se puede vivir en el paraíso de los tontos durante un tiempo, y tal vez llegar a implantar un sistema que incorpore el programa defectuoso, pero tarde o temprano el fallo se hará manifiesto. Si las condiciones que provocan que el fallo se manifieste son fácilmente reproducibles, suele ser relativamente fácil localizar y reparar el problema. Si las condiciones que provocan el fallo no están claras, la vida es mucho más difícil.

Los programas que fallan de forma encubierta suelen ser muy peligrosos. Como no son aparentemente problemáticos, la gente los utiliza y confía en que hagan lo correcto. Cada vez más, la sociedad confía en el software para realizar cálculos críticos que están más allá de la capacidad de los humanos para llevarlos a cabo o incluso comprobar su corrección. Por lo tanto, un programa puede proporcionar una respuesta falaz no detectada durante largos periodos de tiempo. Un

programa que evalúa el riesgo de una cartera de bonos hipotecarios y escupe con confianza una respuesta errónea puede meter a un banco (y quizá a toda la sociedad) en un buen lío. Un programa informático en un ordenador de gestión de vuelos puede marcar la diferencia entre que un avión siga en el aire o no. Una máquina de radioterapia que administra un poco más o un poco menos de radiación de lo previsto puede ser la diferencia entre la vida y la muerte para una persona con cáncer. Un programa que comete un error encubierto sólo ocasionalmente puede o no causar menos estragos que otro que lo comete siempre. Los errores encubiertos e intermitentes son casi siempre los más difíciles de encontrar y solucionar.

Aprendiendo a debugear

La depuración es una habilidad que se aprende. Nadie lo hace bien por instinto. La buena noticia es que no es difícil de aprender y es una habilidad transferible. Las mismas habilidades que se emplean para depurar software pueden servir para averiguar qué falla en otros sistemas complejos, por ejemplo, experimentos de laboratorio o seres humanos enfermos.

Durante al menos cuatro décadas la gente ha estado construyendo herramientas llamadas **depuradores**, y las herramientas de depuración están integradas en todos los **IDEs** populares de Python. Estas herramientas pueden ayudar. Pero lo que es mucho más importante es cómo enfocas el problema. Muchos programadores experimentados ni siquiera se molestan en usar herramientas de depuración, confiando en su lugar en la sentencia `print`.

La depuración comienza cuando las pruebas han demostrado que el programa se comporta de forma no deseada. La depuración *es el proceso de buscar una explicación a ese comportamiento*. La clave para ser consistentemente bueno en la depuración es ser sistemático en la realización de esa búsqueda.

Empiece por **estudiar los datos disponibles**. Esto incluye los resultados de las pruebas y el texto del programa. Estudie todos los resultados de las pruebas. Examine no sólo las pruebas que revelaron la presencia de un problema, sino también las que parecían funcionar perfectamente. Intentar comprender por qué una prueba funcionó y otra no suele ser esclarecedor. Cuando examine el texto del programa, tenga en cuenta que no lo entiende completamente. Si lo hicieras, probablemente no habría ningún fallo.

A continuación, **formule una hipótesis** que considere coherente con todos los datos. La hipótesis podría ser tan limitada como «**si cambio la línea 403 de $x < y$ a $x \leq y$, el problema desaparecerá**» o tan amplia como «**mi programa no funciona porque me olvidé de la posibilidad de aliasing en varios lugares**».

A continuación, **diseñe y ejecute un experimento repetible que pueda refutar la hipótesis**. Por ejemplo, podrías poner una sentencia `print` antes y después de cada bucle. Si éstas siempre están emparejadas, entonces la hipótesis de que un bucle está causando la no terminación ha sido refutada. Decida antes de ejecutar el experimento cómo interpretaría los distintos resultados posibles.

Todos los humanos estamos sujetos a lo que los psicólogos llaman **sesgo de confirmación**: **interpretamos la información de forma que refuerce lo que queremos creer**. Si esperas hasta después de realizar el experimento para pensar en cuáles deberían ser los resultados, es más probable que caigas presa de las ilusiones.

Por último, **lleva un registro de los experimentos que has probado**. Cuando has pasado muchas horas cambiando el código intentando localizar un fallo escurridizo, es fácil olvidar lo que ya has probado. Si no tienes cuidado, puedes perder demasiadas horas probando el mismo experimento (o más probablemente un experimento que parece diferente pero que te dará la misma información) una y otra vez. Recuerda, como muchos han dicho, «**locura es hacer lo mismo, una y otra vez, pero esperar resultados diferentes**».

Diseñando el experimento

Piense en la depuración como un **proceso de búsqueda**, y en cada experimento como un intento de reducir el tamaño del espacio de búsqueda. Una forma de reducir el tamaño del espacio de búsqueda es diseñar un experimento que pueda utilizarse para decidir si una región específica del código es responsable de un problema descubierto durante las pruebas. Otra forma de reducir el espacio de búsqueda es reducir la cantidad de datos de prueba necesarios para provocar la manifestación de un fallo.

Veamos un ejemplo ficticio para ver cómo depurarlo. Imagina que has escrito el código de comprobación de palíndromos.

```

1  def is_pal(x):
2      """Assumes x is a list
3          Returns True if the list is a palindrome; False otherwise"""
4      temp = x[:]
5      temp.reverse()
6      return temp == x
7
8  def silly(n):
9      """Assumes n is an int > 0
10         Gets n inputs from user
11         Prints "Yes" if the sequence of inputs forms a palindrome;
12            "No" otherwise"""
13     for i in range(n):
14         result = []
15         elem = input("Enter element:")
16         result.append(elem)
17
18     if is_pal(result):
19         print("Yes")
20     else:
21         print("No")

```

Ahora, imagine que está tan seguro de sus habilidades de programación que publica este código en Internet sin probarlo. Suponga además que recibe un correo electrónico que dice: «*He probado su programa introduciendo las 3.116.480 letras de la Biblia, y su programa imprimió Sí*». Sin embargo, cualquier tonto puede ver que la Biblia no es un palíndromo. ¡Arréglole! (Su programa, no la Biblia).

Podrías probarlo en la Biblia. Pero sería más sensato empezar probándolo en algo más pequeño. De hecho, tendría sentido probarlo en un mínimo no-palíndromo, por ejemplo,

```

1  >>> silly(2)
2  Enter element: a
3  Enter element: b

```

La buena noticia es que falla incluso esta sencilla prueba, así que no tienes que escribir millones de caracteres. La mala noticia es que no tienes ni idea de por qué ha fallado.

En este caso, el código es lo suficientemente pequeño como para que puedas mirarlo fijamente y encontrar el fallo (o fallos). Sin embargo, supongamos que es demasiado grande para hacerlo, y empecemos a reducir sistemáticamente el espacio de búsqueda.

A menudo, la mejor forma de hacerlo es realizar una **búsqueda biseccional**. Encuentre algún punto más o menos a la mitad del código e idee un experimento que le permita decidir si hay algún problema antes de ese punto que pueda estar relacionado con el síntoma. (Por

supuesto, también puede haber problemas después de ese punto, pero normalmente es mejor buscar un problema cada vez). Al elegir ese punto, busque un lugar en el que algunos valores intermedios fáciles de examinar proporcionen información útil. Si un valor intermedio no es lo que esperaba, probablemente hay un problema que ocurrió antes de ese punto en el código. Si todos los valores intermedios parecen estar bien, el fallo probablemente se encuentre en algún punto posterior del código. Este proceso puede repetirse hasta que haya reducido la región en la que se encuentra el problema a unas pocas líneas de código, o a unas pocas unidades si está probando un sistema grande.

Mirando a lo tonto, el punto medio está alrededor de la línea `if is_pal(result)`. Lo obvio es comprobar si el resultado tiene el valor esperado, `["a", "b"]`. Lo comprobamos insertando la sentencia `print(result)` antes de la sentencia `if` en `silly`. Cuando se ejecuta el experimento, el programa imprime `["b"]`, lo que sugiere que algo ha ido mal. El siguiente paso es imprimir el valor resultante aproximadamente a la mitad del bucle. Esto revela rápidamente que el resultado nunca tiene más de un elemento, lo que sugiere que la inicialización del resultado debe moverse fuera del bucle `for`.

El código correcto para `silly` es:

```
1  def silly(n):
2      """Assumes n is an int > 0
3          Gets n inputs from user
4          Prints 'Yes' if the sequence of inputs forms a palindrome;
5              'No' otherwise"""
6
7      result = []
8
9      for i in range(n):
10         elem = input('Enter element: ')
11         result.append(elem)
12
13     print(result)
14
15     if is_pal(result):
16         print('Yes')
17     else:
18         print('No')
```

Probemos eso, y veamos si el resultado tiene el valor correcto después del bucle `for`. Lo tiene, pero desafortunadamente el programa sigue imprimiendo `Yes`. Ahora, tenemos razones para creer que hay un segundo error debajo de la sentencia `print`. Así que, echemos un vistazo a `is_pal`. Insertamos la siguiente línea antes de la sentencia `return`.

```
1 print(temp, x)
```

Cuando ejecutamos el código, vemos que `temp` tiene el valor esperado, pero `x` no. Avanzando en el código, insertamos una sentencia `print` después de la línea de código `temp = x`, y descubrimos que tanto `temp` como `x` tienen el valor `["a", "b"]`. Una inspección rápida del código revela que en `is_pal` escribimos `temp.reverse` en lugar de `temp.reverse()` la evaluación de `temp.reverse` devuelve el método inverso incorporado para listas, pero no lo invoca.

Ejecutamos de nuevo la prueba, y ahora parece que tanto `temp` como `x` tienen el valor `["b", "a"]`. Hemos reducido el error a una línea. Parece que `temp.reverse()` cambió inesperadamente el valor de `x`. Un error de aliasing nos ha atacado: `temp` y `x` son nombres para la misma lista, tanto antes como después de que la lista se invierta. Una forma de arreglar el fallo es sustituir la primera sentencia de asignación en `is_pal` por `temp = x[:]`, que hace una copia de `x`.

La versión correcta de `is_pal` es:

```
1 def is_pal(x):
2     """Assumes x is a list
3     Returns True if the list is a palindrome; False otherwise"""
4     temp = x[:]
5     temp.reverse()
6     return temp == x
```

Cuando las cosas se ponen difíciles

Se dice que **Joseph P. Kennedy**, padre del Presidente de EE.UU. **John F. Kennedy**, dijo a sus hijos: «*Cuando las cosas se ponen difíciles, los difíciles se ponen en marcha*». Pero nunca depuró un programa informático. Esta subsección contiene algunos consejos pragmáticos sobre qué hacer cuando la depuración se pone difícil.

Busque a los sospechosos habituales. Usted ha

- ¿Pasado argumentos a una función en el orden incorrecto?
- ¿Ha escrito mal un nombre, por ejemplo, escribiendo una letra minúscula cuando debería haber escrito una mayúscula?
- ¿No ha reiniciado una variable?
- ¿Probó que dos valores de coma flotante son iguales (`==`) en lugar de casi iguales (recuerde que la aritmética de coma flotante no es la misma que la aritmética que aprendió en la escuela)?

- ¿Comprobó la igualdad de valores (por ejemplo, comparó dos listas escribiendo la expresión `L1 == L2`) cuando quería comprobar la igualdad de objetos (por ejemplo, `id(L1) == id(L2)`)?
- ¿Olvidó que alguna función incorporada tiene un efecto secundario?
- ¿Ha olvidado el `()` que convierte una referencia a un objeto de tipo función en una invocación a una función?
- ¿Ha creado un alias involuntario?
- ¿Cometió algún otro error típico suyo?

Deje de preguntarse por qué el programa no hace lo que usted quiere. En lugar de eso, **pregúntate por qué hace lo que hace**. Esa debería ser una pregunta más fácil de responder, y probablemente será un buen primer paso para averiguar cómo arreglar el programa.

Ten en cuenta que el fallo probablemente no esté donde crees que está. Si lo estuviera, lo habrías encontrado hace tiempo. **Una forma práctica de decidir dónde buscar es preguntarse dónde no puede estar el bug**. Como dijo **Sherlock Holmes**: «*Elimina todos los demás factores, y el que quede debe ser la verdad*».

Intenta explicar el problema a otra persona. Todos tenemos puntos ciegos. El mero hecho de intentar explicar el problema a otra persona te llevará a menudo a ver cosas que has pasado por alto. También puedes intentar explicar por qué el fallo no puede estar en determinados lugares.

No crea todo lo que lee. En particular, no crea en la documentación. Puede que el código no haga lo que sugieren los comentarios.

Deja de depurar y empieza a escribir documentación. Esto te ayudará a enfocar el problema desde otra perspectiva.

Aléjate y vuelve a intentarlo mañana. Esto puede significar que el fallo se solucione más tarde que si te hubieras quedado con él, pero probablemente perderás menos tiempo buscándolo. Es decir, **es posible cambiar latencia por eficiencia**. (Estudiantes, ¡ésta es una excelente razón para empezar a trabajar en conjuntos de problemas de programación antes que después!)

Cuando ha encontrado el bug

Cuando crees que has encontrado un fallo en tu código, la tentación de empezar a codificar y probar una solución es casi irresistible. Sin embargo, **a menudo es mejor hacer una pausa**. Recuerda que el objetivo

no es arreglar un fallo, sino avanzar rápida y eficazmente hacia un programa sin fallos.

Pregúntese si este fallo explica todos los síntomas observados o si es sólo la punta del iceberg. En este último caso, puede que sea mejor solucionar el fallo junto con otros cambios. Supongamos, por ejemplo, que ha descubierto que el fallo se debe a la mutación accidental de una lista. Podrías evitar el problema localmente, quizás haciendo una copia de la lista. Alternativamente, podrías considerar usar una tupla en lugar de una lista (ya que las tuplas son inmutables), quizás eliminando errores similares en otras partes del código.

Antes de hacer cualquier cambio, intenta comprender las ramificaciones del «arreglo» propuesto. ¿Romperá algo más? ¿Introduce una complejidad excesiva? ¿Ofrece la oportunidad de ordenar otras partes del código?

Asegúrate siempre de que puedes volver a donde estabas. *No hay nada más frustrante que darse cuenta de que una larga serie de cambios te ha dejado más lejos del objetivo que cuando empezaste, y no tener forma de volver al punto de partida.* El espacio en disco suele ser abundante. Utilízalo para almacenar versiones antiguas de tu programa.

Por último, si hay muchos errores inexplicables, podrías plantearte si encontrar y arreglar los fallos de uno en uno es siquiera el enfoque adecuado. Quizá sería mejor que pensaras en una forma mejor de organizar tu programa o quizá en un algoritmo más sencillo que sea más fácil de implementar correctamente.

Enfoque defensivo:

- Especificaciones claras para cada función.
- Verificar condiciones de entrada/salida.
- Modularizar el código.

Tipos de prueba:

- **Unit testing:** probar cada función por separado.
- **Regression testing:** probar errores pasados para asegurarse de que no vuelvan.
- **Integration testing:** probar el programa completo.



Estrategias:

- **Black box testing**: probar solo con la especificación (sin mirar el código).
- **Glass box testing**: probar caminos en el código (conociendo su estructura interna).



Errores comunes:

- **Errores de sintaxis**: olvidarse de paréntesis, mal uso de tipos, etc.
- **Errores lógicos**: el código corre pero hace lo incorrecto.



Proceso de depuración:

1. Identificá el error.
2. Aislalo (¿dónde y cuándo ocurre?).
3. Probá hipótesis con `print`.
4. Retest hasta que funcione para todos los casos.



Frase clave de la clase:

🔗 Frase

“No preguntes qué está mal. Preguntate cómo llegaste al resultado inesperado.”

Ejercicio manual de la lectura

Implemente la función que cumpla las siguientes especificaciones:

```
1 def count_sqrts(nums_list):
2     """
3     nums_list: a list
4     Assumes that nums_list only contains positive numbers and that there
5     are no duplicates.
6     Returns how many elements in nums_list are exact squares of elements
7     in the same list, including itself.
```

```
6         """
7         return len([x for x in nums_list if x ** 0.5 in nums_list])
8
9     # Examples:
10    print(count_sqrts([3,4,2,1,9,25])) # prints 3
```